UNIVERSIDAD DEL VALLE DE GUATEMALA

Diseño de lenguajes de programación – CC3071

Arquitecturación del Generador de Analizadores Léxicos

Ian Cumes - 2336

Javier López- 23415

Javier Cifuentes - 23079

Guatemala, 1 de marzo de 2026

Introducción

Este documento presenta el diseño inicial de la Primera Fase de Compilación: la construcción de un **Generador de Analizadores Léxicos (tipo Lex)**.

El objetivo de esta fase es diseñar un sistema capaz de:

- Leer una especificación léxica.
- Procesar expresiones regulares.
- Construir un autómata finito.
- Generar un analizador léxico funcional.
- Reconocer tokens en una cadena de entrada.

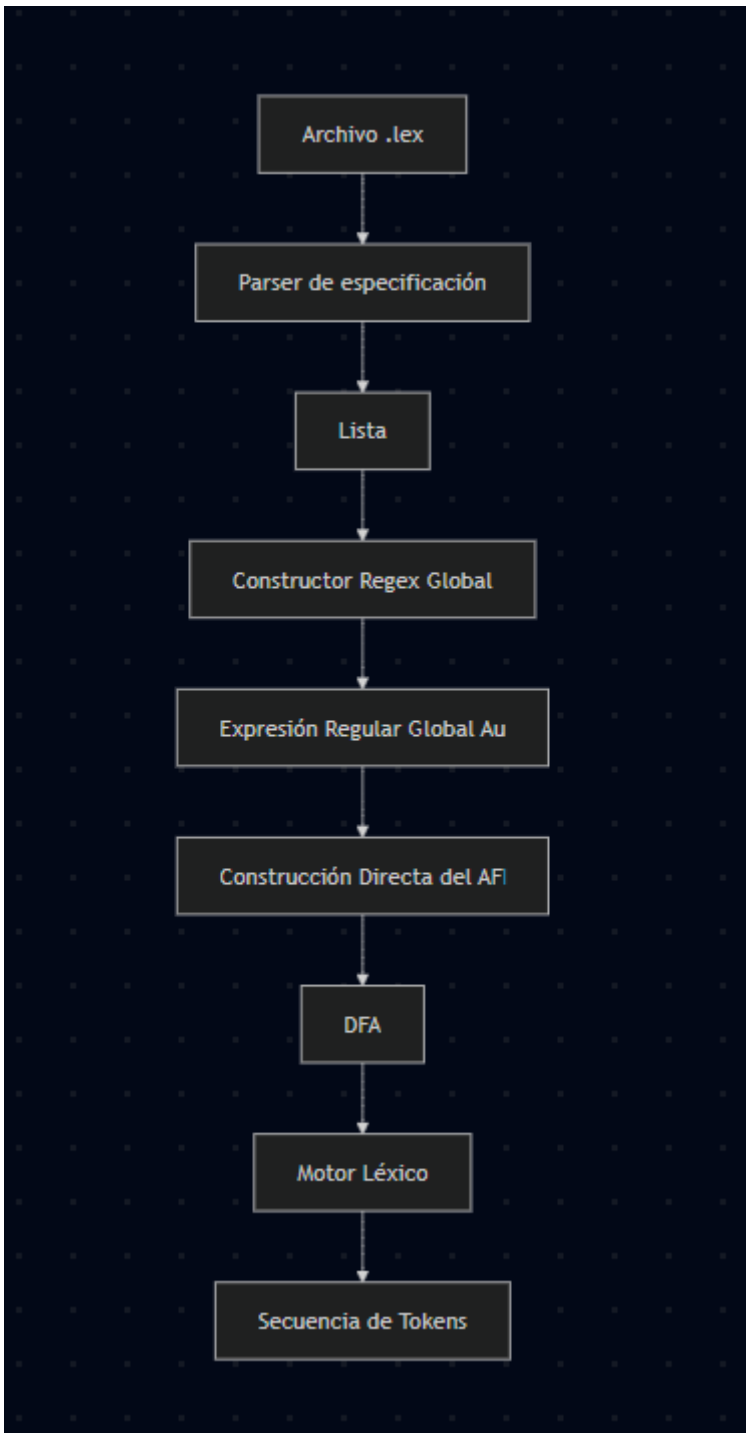
En esta etapa no se implementa el análisis sintáctico; el enfoque es exclusivamente el generador léxico.

Objetivo del Generador Lex

El sistema que diseñaremos deberá:

1. Recibir un archivo de especificación de tokens.
2. Convertir las expresiones regulares en un modelo formal (AFD).
3. Construir el AFD de manera directa
4. Ejecutar el AFD sobre una cadena de entrada.
5. Producir una secuencia de tokens reconocidos.
6. Reportar errores léxicos cuando corresponda.

Arquitectura propuesta



Módulos del Sistema

Parser de Especificación Léxica

Responsabilidad:
Leer el archivo de especificación y extraer las definiciones de tokens.

Ejemplo de entrada:

NUM → [0-9]+
ID → [a-zA-Z][a-zA-Z0-9]*
PLUS → +
MINUS → -

Salida esperada (estructura interna):

```
[
 (NUM, "[0-9]+"),
 (ID, "[a-zA-Z][a-zA-Z0-9]*"),
 (PLUS, "+"),
 (MINUS, "-")
]
```

Este módulo valida sintaxis básica y almacena las expresiones regulares.

Constructor de Expresión Regular Global

Construcción Directa del AFD (Algoritmo followpos)

Objetivo del módulo

Este módulo implementa el algoritmo de construcción directa del AFD a partir de una expresión regular aumentada, utilizando un árbol sintáctico y el cálculo de las funciones nullable, firstpos, lastpos y followpos.

A diferencia del método Thompson + subconjuntos, este método genera directamente un AFD sin pasar por un AFN intermedio.

3.2 Componentes internos que deberán programarse

Este módulo requiere las siguientes piezas programables:

1. Preprocesador de Expresiones Regulares

Responsable de:

- Insertar operador de concatenación explícito (.)
- Agregar símbolo final # a la expresión
- Manejar precedencia de operadores (*, |, .)
- Convertir la expresión a notación postfix (algoritmo shunting-yard)

Salida:

- Lista de tokens en notación postfix

2. Constructor del Árbol Sintáctico

Responsable de:

- Construir el árbol sintáctico a partir de la expresión postfix
- Numerar cada hoja (símbolo terminal)
- Asociar cada hoja con una posición única
- Identificar la hoja correspondiente al símbolo #

Estructura de nodo sugerida:

Nodo:

- tipo (|, ., *, símbolo)
- izquierda
- derecha
- nullable
- firstpos (set de posiciones)
- lastpos (set de posiciones)
- posición (solo en hojas)

Salida:

- Raíz del árbol sintáctico completamente construido

3. Cálculo de Propiedades del Árbol

Para cada nodo se deben calcular:

- nullable
- firstpos
- lastpos

Reglas formales:

Concatenación (A . B):

- nullable = nullable(A) AND nullable(B)
- firstpos = firstpos(A) si no nullable(A), si no union(firstpos(A), firstpos(B))
- lastpos = lastpos(B) si no nullable(B), si no union(lastpos(A), lastpos(B))

Unión (A | B):

- nullable = nullable(A) OR nullable(B)
- firstpos = union(firstpos(A), firstpos(B))
- lastpos = union(lastpos(A), lastpos(B))

Kleene (*):

- nullable = true
- firstpos = firstpos(A)
- lastpos = lastpos(A)

Salida:

- Árbol anotado con estas propiedades

4. Cálculo de Followpos

Se construye una tabla followpos:

Para cada nodo:

- Si es concatenación A . B:
  Para cada posición i en lastpos(A),
  followpos(i) = union(followpos(i), firstpos(B))
- Si es estrella A*:
  Para cada posición i en lastpos(A),
  followpos(i) = union(followpos(i), firstpos(A))

Salida:

- Tabla followpos completa

5. Construcción del AFD

Se construye el AFD directamente:

Estado inicial:

- firstpos(raíz)

Para cada estado S y símbolo a:

- Se construye U = union(followpos(p)) para cada p en S donde símbolo(p) = a
- Si U no existe como estado, se agrega

Un estado es de aceptación si contiene la posición del símbolo #

Salida:

- Conjunto de estados
- Tabla de transición
- Conjunto de estados de aceptación

6. Resolución de Prioridad de Tokens

En caso de múltiples tokens:

- Cada símbolo # debe estar etiquetado con el token correspondiente
- Si un estado contiene múltiples posiciones finales,
  se selecciona el token con mayor prioridad (orden de definición)

Salida:

- Estados de aceptación etiquetados correctamente

Motor de Análisis Léxico

Este módulo ejecuta el AFD sobre la cadena de entrada.

Responsabilidades:

- Leer carácter por carácter.
- Transitar entre estados.
- Aplicar la regla de máximo consumo (longest match).
- Determinar el token correspondiente.
- Detectar errores léxicos.

- Ignorar espacios si están definidos como token descartable.

Salida ejemplo:

(NUM, "12")
(PLUS, "+")
(NUM, "3")

Interacción entre módulos del sistema

Flujo general del sistema

El sistema funciona como una cadena de transformaciones formales donde cada módulo produce una estructura bien definida que sirve como entrada del siguiente módulo.

1. Parser de especificación léxica

Input:

- Archivo .lex

Output:

- Lista<TokenDef>

Cada TokenDef contiene:

- nombre del token
- expresión regular
- prioridad
- flag skip (si aplica)

Este resultado es enviado al módulo siguiente.

2. Constructor de Expresión Regular Global

Input:

- Lista<TokenDef>

Proceso:

- Se combinan todas las expresiones en una sola usando operador |
- Se agregan símbolos finales # etiquetados por token

Ejemplo:

(ID)#1 | (NUM)#2 | (PLUS)#3

Output:

- Expresión regular global aumentada

Esta expresión es enviada al módulo de construcción directa.

3. Construcción Directa del AFD

Input:

- Expresión regular global aumentada

Produce:

- Árbol sintáctico
- Tabla followpos
- Tabla de transición del AFD
- Estados de aceptación etiquetados

Output:

- Objeto DFA completo

Este DFA es enviado al motor léxico.

4. Motor Léxico

Input:

- DFA
- Cadena de entrada

Proceso:

- Simulación determinista
- Aplicación de longest match
- Aplicación de prioridad

Output:

- Lista<Token>
- Lista de errores léxicos

Ejemplo a mano:

Arquitectura Lex

Gramatica de expresiones con identificadores.

Produce tokens

$E \to E+T \mid T$
$T \to id$

Tokens a producir

1 AB → "ab"
2) A → "a"
3 B → "b"

Construir Regex

Regex individuales concatenación explicita =

AB: a·b
A: a
B: b

Marcador aceptación token

AB: a.b.#1
A: a #2
B: b.#3

} Regex global estas 3 con Union "|"

Arbol Sintactico y num hojas

a.b #1: pos1 = a, pos2 = b, pos3 = 1

a.#2: pos: 4 = a, pos5 = #2

b.#3 pos6 = b, pos7 = #3

Tabla posición → Simbolo
1 → a
2 → b
3 → 1 etc.

Calcular nullable, first, last

En hoja Y : Nullable = F
first pos = {pos(x)}
last pos = {pos(x)}

Union X|Y : Nullable = Nullable(X) OR nullable(Y)
first pos (X) U first... (Y)
last = last(X) U last(Y)

Concatenación : null = Null(X) Y Null(Y)
first: first(X) if Not null(X)
if not first(X) U First(Y)

last = last(Y) if null(Y)
then last(X) U last(Y)

Calcular sub-Regex

Concatenar a.b

a: f(1), l(1), n=F
b: f(2), l(2), N=F

Resultado first pos (a.b) = {1},
last pos (a.b) = {2}

Repetir este proceso para

(a.b).#1:
first = {1}, last {3}

Repetir a.#2

Resultado first = 4, last = 5

Repetir b.#3

first = 6 last = 7

Raiz X|Y|Z
first (1,4,6),
last (3,5,7)

Calcular follow pos

Regla para concat

Para todo $i \in$ last pos(λ) =

follow pos(i) += first pos(γ)

Aplicar concat

a.b.#1

follow pos (2) = 3

a.#2

follow pos(4) = 5

b#3

follow pos (6) = 7

Tabla final follow pos

i	Follow pos
1	{2}
2	{3}
3	$\emptyset$
4	{5}
5	$\emptyset$
6	{7}
7	$\emptyset$

Resumen Afd

S_0 {1,4,6} ini

S_1 = {2,5} acepta A

S_2 = {7} acepta B

S_3 = 3 acepta AB

S_d = $\emptyset$

Transiciones

Estado	con a	con b	Acepta
S_0	S_1	S_2	no
S_1	S_d	S_3	si → A
S_2	S_d	S_d	si → B
S_3	S_d	S_d	si → AB
S_d	S_d	S_d	NO

Simular cadena

w = abqbb

escaneo i=0 abqbb

Genera token AB, "ab"

Escaneo i=2

DA: AB con lexema "qb

Ahora i=4

Tokens (AB, "qb") (AB, "qb").

Escaneo i=4

Leo b comision b

Resultado final

(AB, "ab") (AB, "qb") (B, "b")